# CMCS Project 27

Vincenzo Gargano, Lorenzo Pace

January 8, 2025

## Contents

# 1 Problem Statement

(P) is a sparse linear system of the form:

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix (i.e., $D = \text{diag}(D) > 0$ ) and $E \in \mathbb{R}^{(n-1) \times m}$ is obtained by removing the last row from the node-arc incidence matrix of a given connected directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

- (A1) is GMRES, and you must solve the internal problems $\min \left\| H_n y - \|b\| e_1 \right\|$ by updating the QR factorization of $H_n$ at each step: given the QR factorization of $H_{n-1}$ computed at the previous step, apply one more orthogonal transformation to compute that of $H_n$.
- (A2) is the same GMRES, but using the following preconditioner

  $$P = \begin{bmatrix} D^{-\frac{1}{2}} & 0 \\ 0 & R^{-T} \end{bmatrix}$$

  where $S$ is either $S = ED^{-1}E^T$ or a sparse approximation of it (to obtain it, for instance, replace the smallest off-diagonal entries of $S$ with zeros), and $S = R^T R$ is its Cholesky factorization.

No off-the-shelf solvers allowed.

# 2 GMRES

GMRES is an iterative method for computing an approximation of the solution to the linear system $Ax = y$. It accomplishes such approximation by computing:

$$\min_{x \in K_n(A,y)} \|Ax - y\|$$

i.e. by finding the minimum $x$ within the *Krylov Space* $K_n(A, y)$ such that the norm of the residual $Ax - y$ is minimized.

## 2.1 Background: Krylov Spaces and Arnoldi

Given a non-necessarily-symmetric matrix (although in our case it is) $A \in \mathbb{R}^{m \times m}$, a vector $y \in \mathbb{R}^m$, and a natural number $n \leq m$:

$$K_n(A, y) = \text{span } (y, Ay, A^2y, \ldots, A^{n-1}y).$$

We can obtain a good basis for this vector space by using the Arnoldi Algorithm:

$$\texttt{Arnoldi}(A, y, n) =: Q_n, \underline{\mathrm{H}}_n$$

Where:

- $Q_n \in \mathbb{R}^{n \times m}$ is the matrix $(q_1 \mid \ldots \mid q_n)$ of vectors in the basis;
- $\underline{\mathrm{H}}_n \in \mathbb{R}^{n+1 \times n}$ is a matrix such that:

  1. $$\underline{\mathrm{H}}_n = \begin{bmatrix} H_n \\ \ldots \end{bmatrix}$$

     where $H_n \in \mathbb{R}^{n \times n}$ is s.t. $H_n = Q_n^T A Q_n$, and can therefore be seen as $A$ in the Krylov subspace.

  2. $$AQ_n = Q_{n+1}\underline{\mathrm{H}}_n \tag{1}$$

**Remark 1.** *The first vector of the basis generated by* $\texttt{Arnoldi}(A, y, n)$ *is* $\dfrac{y}{\|y\|}$.

## 2.2 Simplifying the minimum problem

We can simplify the minimum problem to:

$$\min_{z \in \mathbb{R}^{n+1}} \|\underline{\mathrm{H}}_n z - e_1 \|y\|\|$$

*Proof.*

$$\begin{aligned}
\min_{x \in K_n(A,y)} \|Ax - y\| &= \min_{z \in \mathbb{R}^{n+1}} \|AQ_n z - y\| && (q_i\text{s form basis of } K_n(A,y)) \\
&= \min_{z \in \mathbb{R}^{n+1}} \|Q_{n+1}\underline{\mathrm{H}}_n z - y\| && (\text{by } (1)) \\
&= \min_{z \in \mathbb{R}^{n+1}} \|Q_{n+1}^T(Q_{n+1}\underline{\mathrm{H}}_n z - y)\| && (Q_{n+1}^T \text{ orthogonal}) \\
&= \min_{z \in \mathbb{R}^{n+1}} \|\underline{\mathrm{H}}_n z - Q_{n+1}^T y)\| && (Q_{n+1} \text{ orthogonal}) \\
&= \min_{z \in \mathbb{R}^{n+1}} \|\underline{\mathrm{H}}_n z - Q_{n+1}^T q_1 \|y\|\| && (\text{by Remark 1}) \\
&= \min_{z \in \mathbb{R}^{n+1}} \|\underline{\mathrm{H}}_n z - e_1 \|y\|\| && (Q_{n+1}^T \text{ orthogonal})
\end{aligned}$$

□

## 2.3 Lanczos

Since the input matrix $A$ is symmetric, we can use the Lanczos Algorithm to iteratively construct the orthonormal basis for the Krylov space. In particular, the $H_n$ matrix resulting from such algorithm will by symmetric and tridiagonal:

$$H_n = \begin{bmatrix} \alpha_1 & \beta_1 & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \beta_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & \beta_{m-2} & \\ & & & \beta_{m-2} & \alpha_{m-1} & \beta_{m-1} \\ & & & & \beta_{m-1} & \alpha_m \end{bmatrix}$$

This allows us to perform the computations without storing the whole matrix, but only the diagonal and subdiagonal. The algorithm is as follows:

**Algorithm 1** Lanczos Algorithm

```
function Lanczos(A, y, n)

    α = β := 0 ∈ ℝ^n          # diagonal and subdiagonal of H
    L := 0 ∈ ℝ^{m×n}          # Lanczos Basis

    # base case
    q := y/‖y‖
    L[:, 1] := q
    w := A*q
    α[1] := w^T * q
    w := w − α[1] * q
    β[1] := ‖w‖

    # inductive case
    for j in 2...n
        if β[j−1] < 1e−13
            println("Breakdown");
            return
        end
        q = w/β[j−1]
        L[:, j] = q

        w := A*q
        α[j] := w^T * q
        w := w − α[j]*q − β[j−1] * L[:, j−1]
        β[j] = ‖w‖

    end

    return (L, α, β)

end
```

## 2.4 Lanczos + QR

A trivial way to implement GMRES would be:

- Use Lanczos to obtain a basis for the Krylov space
- Compute the QR factorization of $\underline{\mathrm{H}}_n$
- Use the factorization to easily solve the linear system arising from the minimum problem.

However, the requested version should compute the QR factorization during the Lanczos iteration.

### 2.4.1 Extending the QR decomposition

Assume we have $\underline{\mathrm{H}}_{n-1} = Q_{old}R_{old}$. The matrix $\underline{\mathrm{H}}_n$ will be computed using a Lanczos iteration, obtaining:

$$\underline{\mathrm{H}}_n = \left[\begin{array}{ccc|c} & \underline{\mathrm{H}}_{n-1} & & \begin{matrix}0\\ \vdots \\ 0\\ \beta_{n-1}\\ \alpha_n\end{matrix} \\ \hline 0 & \dots & 0 & \beta_n \end{array}\right]$$

We want to find two matrices $Q$, $R$ such that:

- $Q \in \mathbb{R}^{(n+1)\times(n+1)}$ is orthogonal
- $R \in \mathbb{R}^{(n+1)\times n}$ is upper triangular
- $\underline{\mathrm{H}}_n = QR$, i.e. $Q^T\underline{\mathrm{H}}_n = R$

First off, to obtain a $Q$ matrix of the requested shape that is orthogonal, we could consider:

$$Q'_{old} = \left[\begin{array}{ccc|c} & Q_{old} & & \begin{matrix}0\\ \vdots \\ 0\end{matrix} \\ \hline 0 & \dots & 0 & 1 \end{array}\right]$$

This takes us close to the solution, but not quite there; in fact the product $Q'^T_{old}\underline{\mathrm{H}}_n$ is *almost* triangular:

$$Q'^T_{old}\underline{\mathrm{H}}_n = \left[\begin{array}{ccc|c} & R_{old} & & Q'^T_{old}\begin{bmatrix}0\\ \vdots \\ 0\\ \beta_{n-1}\\ \alpha_n\end{bmatrix} \\ \hline 0 & \dots & 0 & \beta_n \end{array}\right]$$

Recall that $R_{old}$ is an $n \times (n-1)$ upper triangular matrix, hence it has a row of zeros at the bottom. To make this matrix triangular, we should have a zero in place of the $\beta_n$. We can get rid of such zero by performing a Householder reflection on the bottom two elements of the final column. Let these be:

$$x = \begin{bmatrix} a \\ \beta_n \end{bmatrix} \qquad \mapsto \qquad y = \begin{bmatrix} \|x\| \\ 0 \end{bmatrix}$$

As seen in class, the transformation we need is a reflection w.r.t $v = x - y$:

$$H_{refl} = I - \frac{2vv^T}{\|v\|^2}$$

In order for the reflection to only act on the two bottom elements of the final row, we extend it as follows, obtaining another orthogonal transformation:

$$O = \left[\begin{array}{c|cc} & 0 & 0 \\ I & \vdots & \vdots \\ & 0 & 0 \\ \hline 0 \dots 0 & \multicolumn{2}{c}{\multirow{2}{*}{$H_{refl}$}} \\ 0 \dots 0 & & \end{array}\right]$$

The matrix we obtain as $OQ'^T_{old}\underline{\mathrm{H}}_n$ is triangular.

**Constructing the $R$ Matrix**

- Get old $R$;
- Compute:
$$Q'^T_{old}[0, \dots, 0, \beta_{n-1}, \alpha_n]^T = ([0, \dots, 0, \beta_{n-1}, \alpha_n]Q'_{old})^T$$
Let $q_1 ... q_n$ be the columns of $Q'_{old}$. The result is:
$$\sum_i \beta_{n-1} q_{i,n-1} + \alpha_n q_{i,n}$$
This can be rewritten, letting $r_1 ... r_n$ be the rows of $Q'_{old}$, as:
$$\beta_{n-1} r_{n-1} + \alpha_n r_n$$
- Perform the $O$ transformation to obtain $R$.

**Constructing the $Q$ matrix**

- Get old $Q$;
- Construct $Q'_{old}$ by adding the extra row and column;
- Compute $Q = (OQ'^T_{old})^T$.

Altenatively we can calculate the new Q as follows:

$$Q = O^T Q'_{old} = OQ_{old}$$

By the symmetry of $O$, which is guaranteed by the symmetry of $H_{refl}$. Computationally, we only need to update the last two columns of $Q$, so we need not construct $O$ explicitly. The following algorithm illustrates our approach:

**Algorithm 2** Construction of $Q$ matrix

```
1  Q[j+1, j+1] := 1.0
2  newcols := 0 ∈ ℝ^((j+1)×2)
3  for i in 1...j+1
4     newcols[i, 1] := Q[i, j] * H_refl[1,1] + Q[i, j+1] * H_refl[2,1]
5     newcols[i, 2] := Q[i, j] * H_refl[1,2] + Q[i, j+1] * H_refl[2,2]
6  end
7  Q[1:j+1, j:j+1] := newcols
```

## 2.5 Solving the minimum problem, given QR

Given the QR factorization of $\underline{\mathrm{H}}_n$, we can transform our minimum problem:

$$\boxed{\min_{z\in\mathbb{R}^{n+1}} \|\underline{\mathrm{H}}_n z - e_1\|y\|\|} \quad \text{into} \quad \boxed{\min_{z\in\mathbb{R}^{n+1}} \|R_0 z - Q_0^T(e_1\|y\|)\|}.$$

*Proof.* Let $\underline{\mathrm{H}}_n = QR, \quad Q = [Q_0, Q_c], \quad R = [R_0, 0]^T$.

$$\begin{aligned}
\|\underline{\mathrm{H}}_n z - e_1\|y\|\| &= \|Q^T(\underline{\mathrm{H}}_n z - e_1\|y\|)\| && (Q^T \text{ is norm-preserving}) \\
&= \|Q^T Q R z - Q^T(e_1\|y\|)\| && (\text{QR factorization of } \underline{\mathrm{H}}_n) \\
&= \left\| \begin{bmatrix} R_0 \\ 0 \end{bmatrix} z - \begin{bmatrix} Q_0^T \\ Q_c^T \end{bmatrix} (e_1\|y\|) \right\| && (\text{assumption}) \\
&= \left\| \begin{bmatrix} R_0 z - Q_0^T(e_1\|y\|) \\ -Q_c^T(e_1\|y\|) \end{bmatrix} \right\| && (\text{vector algebra})
\end{aligned}$$

Whose minimum is only influenced by the top portion (the bottom one does not depend on $z$). □

Once we have transformed the problem to this form, if $R_0$ is invertible (which is true iff the $\underline{\mathrm{H}}_n$ is full-rank), we have obtained a *triangular system*, which is efficiently solved by backsubstitution.

## 2.6 Computational Complexity

### 2.6.1 Time Complexity of Lanczos+QR

The algorithm consists of a for-loop of length $n$ (the number of Lanczos iterations). We go through the dominating parts of the $j$-th step of the loop:

- The Lanczos iteration's complexity, both in its base and inductive cases, is dominated by the matrix-vector multiplication $w := Aq$, which has time complexity $O(m^2)$ ($m$ is the size of $A$).
- The base case of the QR construction consists of a $2 \times 2$ QR factorization, which can easily be considered as a constant-time operation.
- The inductive case of the QR construction is dominated by the $O(j)$ cost of computing the new columns of the $Q$ and $R$ matrices.

This yields a time complexity of

$$O\left(nm + \frac{n(n+1)}{2}\right) = O(nm + n^2)$$

Taking in account the fact that we use a sparse matrix data type in the implementation, we can express the time complexity as follows:

$$O(\texttt{nnz(A)} + n^2)$$

### 2.6.2 Space Complexity of Lanczos+QR

Let $\#D$ represent the length of $D$'s diagonal. We need to store:

- The matrix (in sparse form): $O(\#D + 2 \cdot \texttt{NNZ(E)})$
- The Lanczos basis: $O(m \cdot n)$
- $Q$ and $R$: $O(n^2)$

Hence the space complexity is the sum of the above complexities.

### 2.6.3 Time complexity of GMRES

Our implementation of GMRES consists of:

- A Lanczos+QR computation $O(\texttt{nnz(A)} + n^2)$
- Solving an $n \times n$ triangular system: $O(n^2)$

Hence the complexity is dominated by the Lanczos+QR computation $O(\texttt{nnz(A)} + n^2)$.

### 2.6.4 Space complexity of GMRES

Once again, the space complexity is dominated by that of Lanczos+QR, it is therefore:

$$O(\#D + 2 \cdot \texttt{NNZ(E)} + mn + n^2)$$

## 2.7 Stability

The three components of our method are the Lanczos method, the QR factorization and the back substitution used to solve the triangular linear system resulting from the QR factorization.

- **Lanczos** As remarked in [6], *"no straightforward theorem is known to the effect that the Lanczos iteration is stable in the sense defined in this book"*.
- **QR** As for the QR factorization, we provide a proof of its backward stability.
- **Back substitution** A proof of the backward stability of back substitution can be found in [3].

### 2.7.1 Background: Backward Stability

An algorithm is said to be *backward stable* if the computed output $\widetilde{y}$ can be written as $\widetilde{y} = f(\widehat{x})$, where:

$$\frac{\|\widehat{x} - x\|}{\|x\|} = O(u)$$

where $\widehat{x}$ is a perturbed input $x + \Delta x$.

### 2.7.2 Backward Stability of QR factorization

First, we recall the following result:

**Theorem 1** (Backward stability of orthogonal transformations)**.** *If $Q \in \mathbb{R}^{m\times m}$ is orthogonal, and $B \in \mathbb{R}^{m\times n}$, then the computed version $\widetilde{C}$ of $C = QB$ is backward stable:*

$$\widetilde{C} = Q\widehat{B}, \qquad \|\widehat{B} - B\| \leq O(u)\|B\|$$

The proof is omitted. We can now prove the following:

**Claim 1.** *Let $A + \Delta_A$ be a perturbed version of the input matrix $A$. We compute:*

$$\widetilde{Q}\widetilde{R} = \texttt{QR}(A + \Delta_A).$$

*The computed $\widetilde{Q}$ and $\widetilde{R}$ satisfy:*

$$\widetilde{Q}\widetilde{R} = A + \Delta \quad \textit{with} \quad \|\Delta\| \leq O(u)\|A\|.$$

*Proof.* Recall that:

$$A_0 = A, \qquad A_k = Q_k \dots Q_1 A, \qquad k \in \{1..n\}$$

Where the $Q_k$s are orthogonal matrices of the form:

$$\begin{bmatrix} I & 0 \\ 0 & H_k \end{bmatrix}$$

And $H_k$ is a Householder reflector matrix. We first show by induction that:

$$\widetilde{A}_k := \widetilde{Q}_k(\widetilde{A}_{k-1} + \Delta)$$

is such that $\|\Delta\| \leq O(u)\|A\|$ and $\|\widetilde{A}_k\| \leq \|A\| + O(u)\|A\|$.

- **Base case** By Theorem 1:

$$\widetilde{A}_1 = \widetilde{Q}_1(A + \Delta_1), \qquad \|\Delta_1\| \leq O(u)\|A\|$$

Additionally:

$$\begin{aligned} \|\widetilde{A}_1\| &= \|\widetilde{Q}_1(A + \Delta_1)\| \\ &= \|A + \Delta_1\| && \text{(orthogonality)} \\ &\leq \|A\| + O(u)\|A\| && \text{(prop. of norms, above ineq.)} \end{aligned}$$

- **Inductive Case** Let us assume that $\widetilde{A}_i = \widetilde{Q}_i(\widetilde{A}_{i-1} + \Delta_i)$, with:

$$\text{(Hp 1)} \quad \|\Delta_i\| \leq O(u)\|A\| \qquad \text{(Hp 2)} \quad \|\tilde{A}_i\| \leq \|A\| + O(u)\|A\|.$$

Then:

$$\widetilde{A}_{i+1} = \widetilde{Q}_{i+1}(\widetilde{A}_i + \Delta_{i+1})$$

We thus have:

$$\begin{aligned} \|\Delta_{i+1}\| &\leq O(u)\|(\widetilde{A}_i + \Delta_i)\| && \text{(by Theorem 1)} \\ &\leq O(u)(\|\widetilde{A}_i\| + \|\Delta_i\|) && \text{(prop. of norms)} \\ &= O(u)(\|\widetilde{A}_i\| + O(u)\|A\|) && \text{(Inductive Hp 1)} \\ &\leq O(u)(\|A\| + O(u)\|A\| + O(u)\|A\|) && \text{(Inductive Hp 2)} \\ &= \|A\|O(u) \end{aligned}$$

And finally:

$$\begin{aligned} \|\widetilde{A}_{i+1}\| &= \|\widetilde{Q}_{i+1}(\widetilde{A}_i + \Delta_{i+1})\| \\ &= \|\widetilde{A}_i + \Delta_{i+1}\| && \text{(Orthogonality)} \\ &\leq \|\widetilde{A}_i\| + \|\Delta_{i+1}\| && \text{(prop. of norms)} \\ &\leq \|A\| + O(u)\|A\| + \|\Delta_{i+1}\| && \text{(Inductive Hp 2)} \\ &\leq \|A\| + O(u)\|A\| + \|A\|O(u) && \text{(above inequality)} \\ &= \|A\| + O(u)\|A\| \end{aligned}$$

Now recall that $\widetilde{R} = \widetilde{A}_k$ for some $k$, and therefore, given:

$$\widetilde{Q}\widetilde{R} = A + \Delta$$

we can conclude:

$$\begin{aligned} \|\Delta\| &\leq O(u)\|A_k\| && \text{(by Theorem 1)} \\ &\leq O(u)(\|A\| + O(u)\|A\|) && \text{(as shown by induction)} \\ &= O(u)\|A\| \end{aligned}$$

□

# 3 GMRES with preconditioner

In this section we discuss how we implemented the algorithm using the following preconditioner:

$$P = \begin{bmatrix} D^{-\frac{1}{2}} & 0 \\ 0 & R^{-T} \end{bmatrix}$$

where where $S$ is either $S = ED^{-1}E^T$ or a sparse approximation of it, and $S = R^T R$ is its Cholesky factorization. Since our matrix is symmetric, the preconditioned problem we want to solve is:

$$PAP^T(P^{-T}x) = Py$$

## 3.1 Implementation details

We do not construct the preconditioner explicitly. Rather, we perform multiplications by $PAP^T$ as:

```
w = mult_by_PT(q, sqrtD, Uinv)
w = A * w
w = mult_by_P(w, sqrtD, Uinv)
```

Where `sqrtD` is $D^{-1/2}$, `Uinv` is $U^{-1}$, and:

```
function mult_by_P(v, sqrtD, Uinv)
    sizeD = size(sqrtD)[1]
    sizeP = sizeD + size(Uinv)[1]

    bu = v[1 : sizeD]
    bb = v[sizeD + 1 : sizeP]
    v = [sqrtD * bu ; Uinv' * bb]

    return v
end

function mult_by_PT(v, sqrtD, Uinv)
    return mult_by_P(v, sqrtD, Uinv')
end
```

**Approximation of $S$** Once $S$ is computed, we can remove all off-diagonal elements under some threshold using the following Julia code:

```
  # set off-diagonal elements under some threshold to zero
  if THRESHOLD_ZERO > 0
      threshold = THRESHOLD_ZERO
      mask = .!I(heightE) .& (abs.(S) .< threshold)
      S[mask] .= 0
  end
```

### 3.1.1 Complexity and stability

The results for the computational complexity and stability of the algorithm remain unchanged from those stated previously, for the algorithm without preconditioning. The one difference that had an impact on performance was the increased density of the preconditioned matrices, which makes it so that the preconditioned version of the algorithm takes longer to compute.

# 4 Additional implementations

## 4.1 Restarted versions of the algorithms

In addition to the GMRES implementation, we provided a *higher-order function* that takes as input any GMRES implementation, and uses it to perform the **restarted GMRES** algorithm, proposed as an exercise in chapter 35 of [6] and further discussed in [5].

**Reasons for using the restarted version** For larger values of $n$, the cost of GMRES in operations and storage may be prohibitive. This is because, as seen before, the algorithm is quadratic both in time and in space. One way to mitigate these issues is to *restart* the algorithm after a certain number of iterations. The restarted GMRES algorithm, as implemented in Julia, is as follows:

```
function GMRES_Restarted(GMRES_Method :: Function, A::SparseMatrixCSC, y::
    Vector, args :: Tuple, max_iter::Int, restart::Int, tol::Float64)
    x = spzeros(size(A, 2))

    # Current residual
    r = y - A * x
    norm_r = norm(r)

    iter = 0
    while norm_r > tol && iter < max_iter

        # Perform GMRES for 'restart' iterations
        dx = GMRES_Method(args..., Vector(r), restart)

        # Update solution and residual
        x += dx
        r = y - A * x
        norm_r = norm(r)

        iter += restart

        println("Iteration $iter, Residual Norm: $norm_r")
    end

    if norm_r <= tol
        println("Converged in $iter iterations with residual norm $norm_r")
    else
        println("Did not converge within $max_iter iterations. Final residual
    norm: $norm_r")
    end

    return x
end
```

**Correctness: restart invariant** Let $r_i$ be the residual after restart $i$: $r_i = y - Ax_i$. We have the following:

$$Ax_i = y - r_i$$

$$x' := \texttt{GMRES}(A, r_i) \implies Ax' = \widetilde{r}_i \approx r_i$$

Let $x_{i+1} = x_i + x'$, now:

$$A(x_{i+1}) = Ax_i + Ax' = y - \widetilde{r}_i + r_i \approx y$$

This shows that, at each restart:

$$Ax_i \approx y.$$

Indeed, we should ensure that, under some assumptions, the approximations grow in precision with each iteration, but we do not go into this: with the right parameter choice, we found experimentally that the method provides very good approximations of the solution in very brief time.

**Drawbacks of the method** A well known difficulty with the restarted GMRES algorithm is that it can stagnate when the matrix is not positive definite [5]. Indeed, experimentally, we have noticed that choosing the wrong parameters (`restart` and tolerance) would yield stagnation. However, as evident from the experimental data, the restarted algorithm often delivered solutions orders of magnitude more fast and precise than the bare implementations.

# 5 Generating instances of the problem

## 5.1 Min-Cost Flow

The problem arises from the KKT system of the convex quadratic separable Min-Cost Flow Problem. The problem is defined as follows:

- Let $G = (V, A)$ be a graph, with $V$ its set of nodes, and $A$ its set of arcs.
- Let $b \in \mathbb{R}^{|V|}$ be a vector of *node flows* (based on which we can classify nodes as source, target or transshipment).
- Let $x \in \mathbb{R}^{|A|}$ be a *flow assignment*:

$$x = \begin{bmatrix} \varphi_1 \\ \vdots \\ \varphi_{|A|} \end{bmatrix}$$

- Let $\varphi_{ij}$ be the flow on the edge $(i, j) \in A$.
- Let $C_{ij} : \mathbb{R} \to \mathbb{R}$ be a quadratic function that maps a flow $\varphi$ to the *cost* of transporting such flow through the $(i, j)$ edge.

We formulate the problem as follows:

$$\min \sum_{(i,j) \in E} C_{ij}(\varphi_{ij})$$

With the constraint given by Kirchhoff's law of conservation of flow:

$$\forall i \in V. \sum_{\substack{i \in V \\ (j,i) \in A}} \varphi_{i,j} - \sum_{\substack{i \in V \\ (i,j) \in A}} \varphi_{i,j} = b_i.$$

There would be other constraints, e.g. the non-negativity of flows or per-edge upper and lower bounds to the flows, but in order to obtain a KKT system of the requested shape we do not consider these.

## 5.2 Problem and constraints in matrix form

We mentioned that each function $C_{ij}(\varphi)$ is quadratic. Let us write it as:

$$\forall e \in A \,.\, C_e(\varphi) = \frac{1}{2} c_e \varphi^2 + d_e \varphi$$

We can thus rewrite the minimum problem as:

$$\min \left( x^T D x + \langle d, x \rangle \right)$$

Where:

$$D = \begin{bmatrix} c_1 & & 0 \\ & \ddots & \\ 0 & & c_{|A|} \end{bmatrix} \qquad d = \begin{bmatrix} d_1 \\ \vdots \\ d_{|A|} \end{bmatrix}$$

Additionally, given the incidence matrix $\underline{E}$, we can write the constraint as simply $\underline{E}x = b$.

## 5.3 KKT System

From the above problem and equality constraint, we obtain the following system:

$$\underline{E}x = b \tag{KKT-F}$$

$$Dx + \underline{E}^T \cdot \mu = -d \tag{KKT-G}$$

With $\mu \in \mathbb{R}$. The system can be rewritten as:

$$\begin{bmatrix} D & \underline{E}^T \\ \underline{E} & 0 \end{bmatrix} \begin{bmatrix} x \\ \mu \end{bmatrix} = \begin{bmatrix} -d \\ b \end{bmatrix}$$

In order to obtain an equivalent full-rank system, as suggested in the problem statement, we can remove one row from the incidence matrix.

## 5.4 Netgen dataset, and generating $D$

In order to obtain realistic instances to test our algorithms, we used the netgen [4] dataset. This provides graphs in the DIMACS format defined in [1], coming with the following information:

- Node flow of each vertex
- Source, target, capacity, and *linear* cost of each edge.

The main issue with this dataset is that, indeed, it only provides linear costs.

**Generating $D$** Frangioni et al. [2] propose a procedure for generating realistic quadratic costs. Their approach is to generate the costs uniformly in $[ac_{ij}, bc_{ij}]$, where $c_{ij}$ are the linear costs and $a, b$ are integer numbers. The interval considered in the paper was $[3c_{ij}, 10c_{ij}]$.

**Other ways to construct $D$** In order to further explore the performance of our algorithm, we will additionally test with other versions of the matrix $D$, namely:

- Diagonal with all ones
- Diagonal with entries uniformly distributed in $[0, 1]$

# 6 Experimental Results

The following experimental results refer to a 9215×9215 matrix, and were ran on a laptop with a *13th Gen Intel(R) Core(TM) i5-1335U* processor, and 16 GB of RAM.

## 6.1 Comparison in performance for different $D$

As outlined in Figure 1, we obtained the following results:

- With $D = I$, the algorithm reaches convergence to $10^{-10}$ in fewer than 200 steps
- With the uniform distribution, the algorithm settles on $10^{-8}$ in almost 1500 steps.
- The matrix with the generated quadratic costs is quite ill-conditioned, with a condition number of $1.347 \cdot 10^{11}$. Therefore, the algorithm does not quite converge at all.

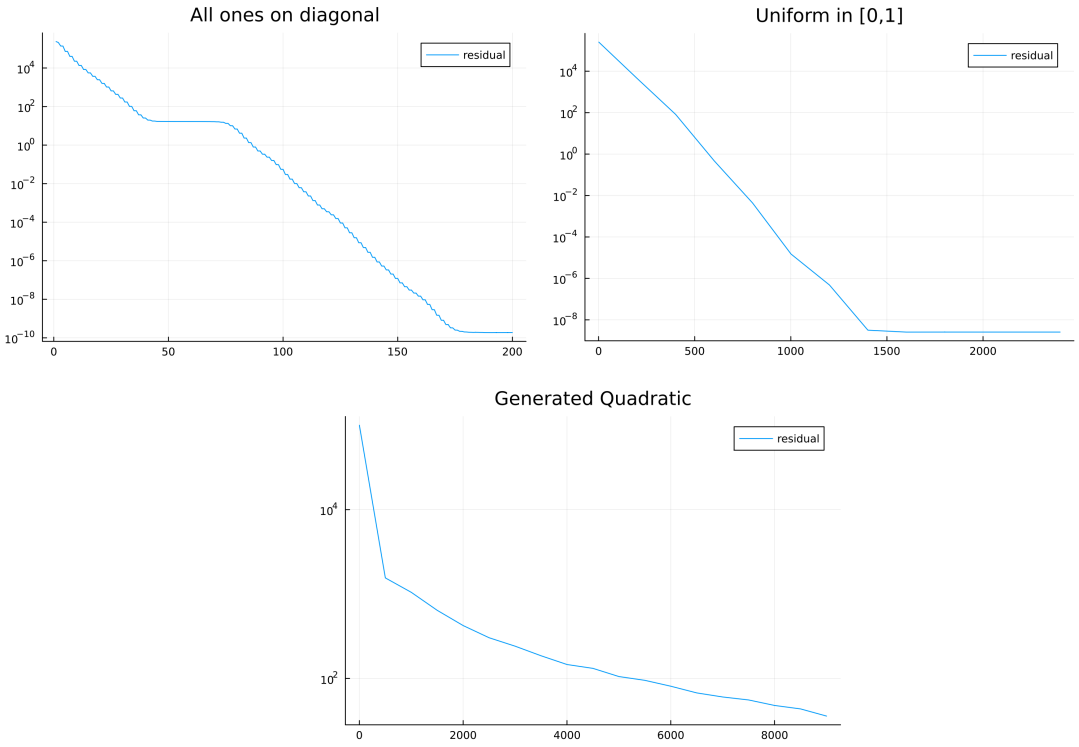


Figure 1: Residuals vs iterations for different constructions of $D$.

## 6.2 Comparison between the precond. and non-precond. systems

In this section we compare the preconditioned algorithm to the non-preconditioned one. In Figure 2, we show how:

- For the matrix with $D = I$, we remark that the preconditioned algorithm converges much faster, albeit to a slightly less precise result.
- The same is true for the matrix with elements in $[0, 1]$.

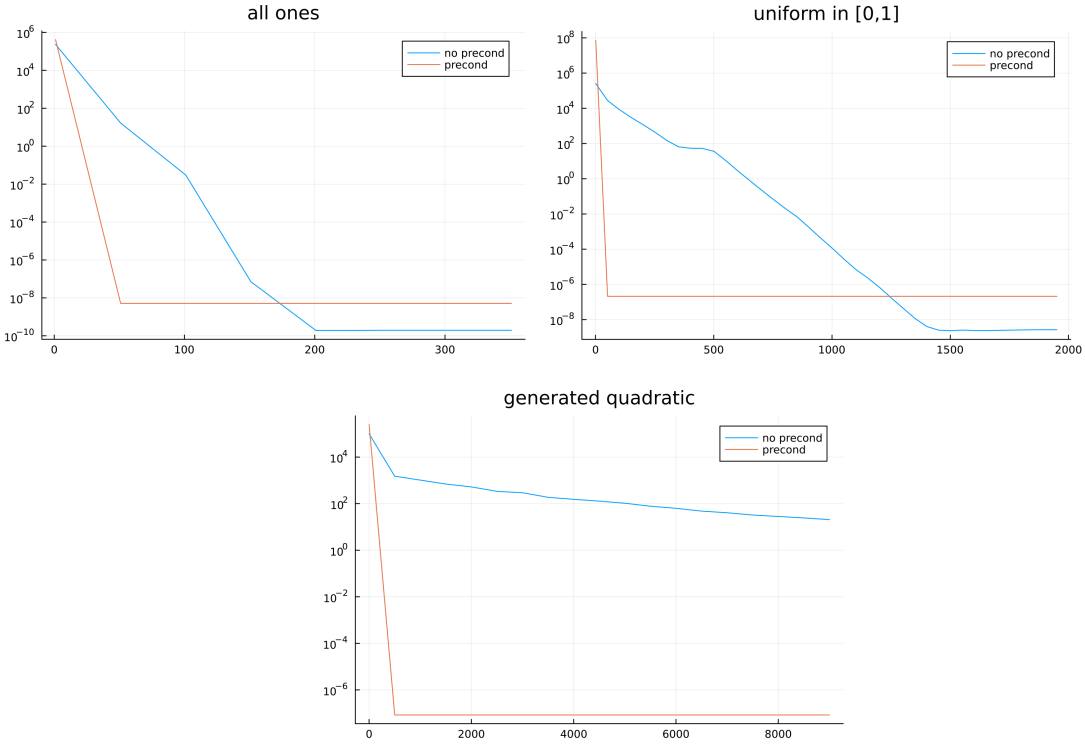


Figure 2: Comparison between preconditioned and non-preconditioned system.

- The preconditioned algorithm performs the best with the ill-conditioned matrix obtained generating the quadratic weights. Not only does it converge fast, but the result is several orders of magnitude more precise than the non-preconditioned algorithm.

From these observations, in particular the one regarding the ill-conditioned matrix, we conclude that the preconditioner works. The reason why the performance is not improved in the first two cases, is that the two matrices were already well-conditioned, as shown in the following table.

| | **Non-precond.** | **Precond.** |
|---|---|---|
| **All ones** | $7.104 \cdot 10^3$ | 2.618 |
| **Uniform in** $[0, 1]$ | $2.74 \cdot 10^3$ | 2.618 |
| **Generated Quadratic** | $1.347 \cdot 10^{11}$ | 2.618 |

The value of the condition number for all three preconditioned matrices, $\sim$2.61803398875, appears to be the *golden ratio* ($\varphi$) plus one.

## 6.3 Comparison between GMRES and Restarted GMRES

As mentioned, for the correct choice of the `restart` parameter, the restarted version of the algorithm is able to provide faster and more precise results. In figures 3,4,5 we compare all the methods described in this report, including the restarted ones. Once again, we used a 9215×9215 matrix from the `netgen` dataset, on the same setup as above.

### 6.3.1 Matrix with $D = I$

With this particularly well-behaved matrix, all proposed methods converge very fast, the fastest one being the restarted method (with `restart` = 300), and the most precise being the restarted preconditioned method, which reaches $10^{-10}$. The convergence times are all well below one second.

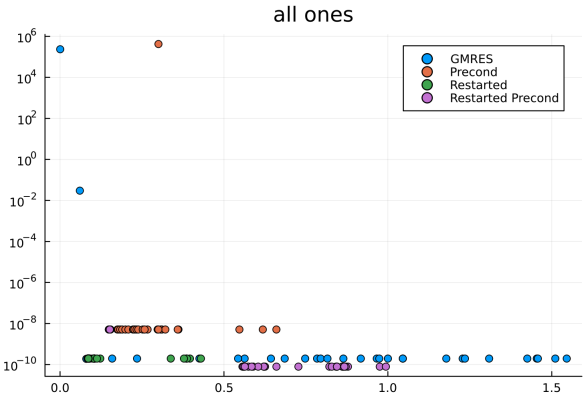


Figure 3: Comparison of all methods for $D = I$.

### 6.3.2 Matrix with diagonal in [0,1]

With this matrix, the fastest converging method is the preconditioned algorithm, which reaches convergence in less than one second. The most precise algorithm is still the restarted preconditioned algorithm (with `restart` = 300), which reaches a precision of $10^{-10}$ in less than two seconds.

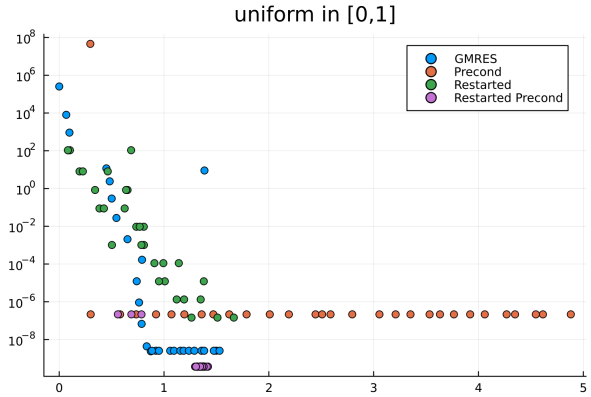


Figure 4: Comparison of all methods for $D$ uniform in $[0, 1]$.

### 6.3.3 Matrix with generated quadratic costs

With this ill-conditioned matrix, GMRES and Restarted GMRES do not reach convergence. The preconditioned algorithm converges to $10^{-7}$ in less then one second, whereas the restarted preconditioned (with `restart` $= 300$) algorithm reaches $10^{-10}$ in less than two seconds.

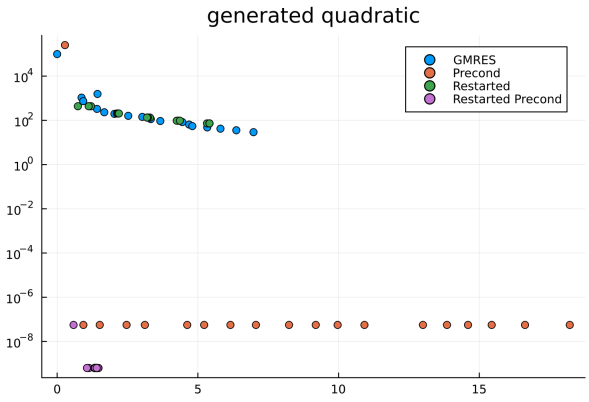


Figure 5: Comparison of all methods for generated quadratic costs.

# 7 Command Line Tool

We wrote a command line tool to facilitate the testing of our algorithms. The following is its usage, which can be obtained by writing `julia -p 1 main.jl --help`

```
usage: main.jl [--iterations ITERATIONS] [--restart RESTART]
               [--restart_precond RESTART_PRECOND]
               [--diagonal DIAGONAL] [--tol TOL] [--step STEP]
               [--precond] [--restarted] [--nworkers NWORKERS] [-h]
               mode data

positional arguments:
  mode                  Program mode. Can be "compute" or "bench".
  data                  Path to the problem data.

optional arguments:
  --iterations ITERATIONS
                        Number of iterations. (type: Int64, default:
                        1000)
  --restart RESTART     Restart parameter of restarted algorithm.
                        (type: Int64, default: 300)
  --restart_precond RESTART_PRECOND
                        Restart parameter of restarted preconditioned
                        algorithm. (type: Int64, default: 300)
  --diagonal DIAGONAL   Can be "ones", "uniform" or "quadratic"
                        (default: "quadratic")
  --tol TOL             Tolerance parameter of restarted algorithm.
                        (type: Float64, default: 1.0e-9)
  --step STEP           Step between iter values in bench mode. If 0,
                        uses iterations/4 (type: Int64, default: 0)
  --precond             Preconditioned? [used in compute mode].
  --restarted           Restarted? [used in compute mode].
  --nworkers NWORKERS   Number of workers for the conversion function.
                        Should be used in tandem with "julia -p
                        nworkers" (type: Int64, default: 1)
  -h, --help            show this help message and exit
```

## 7.1 Data generation

We provide a parser (`parser_matrix.py`) that transforms the `dmx` files from the netgen dataset to `npz` compressed archives. In the dataset directory, we include some compressed data representing graphs of increasing size and density.

## 7.2 Conversion function

In order to have the data occupy less space, we used the `npz` data format to store them. However, this is not directly compatible with the Julia sparse matrix format, therefore to load the incidence matrices from disk we first run them through a conversion function we wrote.

We provide two versions of the conversion function: a serial one and a parallel one.

- For incidence matrices that are small enough, the serial converter completes the conversion faster than the parallel one. The conversion times are usually around 1 or 2 seconds for this kind of matrices.
- For larger matrices, such as the one obtained from: `net12_64_1.dmx_out.npz`, the serial algorithm is way too slow. In these cases, it is necessary to use the parallel one.

## 7.3 Examples

In order to simplify the usage of the tool, we provide a **makefile**, with two targets:

- **run**, which performs the serial conversion:

```
make run ARGS="compute ./dataset/net10_8_1.dmx_out.npz --precond --
  iterations 200"

```

- **run_mp**, which stands for "multi-process", and performs the parallel conversion:

```
make run_mp ARGS="compute ./dataset/net12_64_1.dmx_out.npz --precond --
  iterations 200" NWORKERS="10"

```

**Modes** The command line tool has two "modes". **Compute mode** can be used to perform the algorithm on a graph from the dataset.

```
$ make run ARGS="compute ./dataset/net10_8_1.dmx_out.npz --precond --
    iterations 200"
julia -p 1 main.jl compute ./dataset/net10_8_1.dmx_out.npz --precond --
    iterations 200
converting data format...
converted in 1.581830902 seconds.

Compute mode
============
Performing 200 iterations:
residual: 1.1924300300246332e-7 computed in 0.849061762s

```

On the other hand, **bench mode** can be used to plot the comparison between the methods. In the following example, since the step is not specified through a command line option, it defaults to $\frac{1}{4}$ times the number of iterations.

```
$ make run ARGS="bench ./dataset/net10_8_1.dmx_out.npz --diagonal ones --
    iterations 200"
julia -p 1 main.jl bench ./dataset/net10_8_1.dmx_out.npz --diagonal ones --
    iterations 200
converting data format...
converted in 1.215956481 seconds.

Bench mode
==========
Performing 200 iterations with step = 50
Testing GMRES...
Testing precond GMRES...
 * [...]
Testing restarted GMRES...
 * [...]
Testing restarted precond GMRES...
 * [...]
press enter to terminate program.

```

The program produces the scatter plot in figure 6.

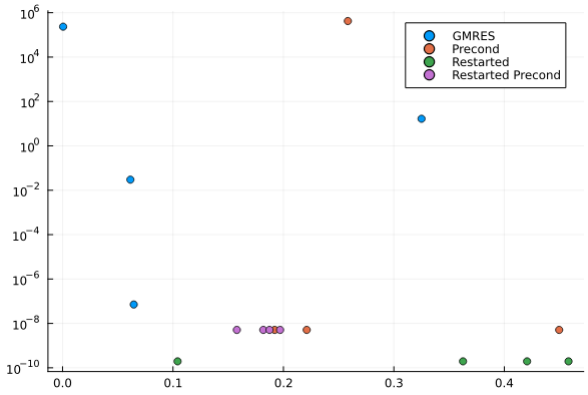


Figure 6: Example plot.

### 7.4 Notes on iterations and memory usage

Since in our algorithm it is necessary to store the $n$-dimensional basis of the Krylov space explicitly in memory, where $n$ is the number of iterations, a high number of iterations for a very large input matrix will result in high memory usage.

### 7.5 Notes on the language

#### 7.5.1 JIT and TTFX

Julia is a dynamic programming language commonly used in numerical analysis. Its most important feature is *just-in-time* (JIT) compilation. In our code, in order to get the timings correct, we had to make sure that each portion of code is compiled before it is timed. We did this by calling it with dummy values before performing the actual timing.

One drawback of JIT compilation is the so-called "time to first execution" (TTFX). Each time we run a portion of a Julia script for the first time, there is a slight delay due to the fact that the code is being compiled.

### 7.6 Dependencies

In Julia, packages are installed from whithin the language itself. The dependencies can be installed by running: `julia requirements.jl`, which will install the packages used in our project.

## 8 Conclusions

We implemented two algorithms, GMRES and preconditioned GMRES, and a higher-order function that provides the *restarted* version of any GMRES implementation. The algorithm without preconditioning is not able to provide satisfying results with realistic, ill-conditioned systems. The preconditioned algorithm is, instead, adequate for the purpose. However, for the right choice of the `restart` parameter, the *restarted preconditioned* algorithm is able to converge quickly to results much more precise than any of the other proposed algorithms, even in the case of ill-conditioned matrices.

# References


[1] *DIMACS minimum cost flow problems*. URL: https://web.mit.edu/lpsolve_v5525/doc/DIMACS_mcf.htm.

[2] Antonio Frangioni et al. "Projected perspective reformulations with applications in design problems". In: *Operations research* 59.5 (2011), pp. 1225–1232.

[3] Nicholas J Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2002.

[4] *NETGEN*. URL: https://commalab.di.unipi.it/datasets/mcf/#LinMCF.

[5] Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.

[6] Lloyd N Trefethen and David Bau. *Numerical linear algebra*. SIAM, 2022.